whole path from code commit to production and achieve continuous deployment.

15.3.2 Implementing the production stage with GitHub Actions

Compared to the previous stages, implementing the production stage of a deployment pipeline can differ a lot depending on several factors. Let's start by focusing on the first step of the production stage.

At the end of the acceptance stage, we have a release candidate that's proven to be ready for production. After that, we need to update the Kubernetes manifests in our production overlay with the new release version. When we're keeping both the application source code and deployment scripts in the same repository, the production stage could be listening to a specific event published by GitHub whenever the acceptance stage completes successfully, much like how we configured the flow between the commit and acceptance stages.

In our case, we are keeping the deployment scripts in a separate repository, which means that whenever the acceptance stage workflow completes its execution in the application repository, we need to notify the production stage workflow in the deployment repository. GitHub Actions provides the option of implementing this notification process via a custom event. Let's see how it works.

Open your Catalog Service project (catalog-service), and go to the acceptance-stage.yml file within the .github/workflows folder. After all the acceptance tests have run successfully, we have to define a final step that will send a notification to the `polar-deployment` repository and ask it to update the Catalog Service production manifests with the new release